

P4-10 AI 协作反思日志 #4

项目： CampusHub
阶段： P4 核心开发与功能完善
团队： CampusHub 小组
日期： 2026-06-06
对应交付物： 10. AI 协作反思日志 #4

1. 本阶段 AI 使用清单

任务	AI 工具	Prompt 摘要	AI 输出质量(1-5)	人工修改幅度
Sprint 合并任务拆分	Codex / ChatGPT	根据 P1-P3 文档拆分 P4 开发任务，标注优先级、负责人、工时和完成标准	4	约 30%
接口任务清单整理	Codex / ChatGPT	根据 P3 API 设计和实际 Controller 整理接口联调清单	4	约 25%
订单状态测试补充	Codex / ChatGPT	为订单完成、取消、评价、消息等规则生成单元测试建议	4	约 40%
集成测试设计	Codex / ChatGPT	设计登录、发布、申请、确认成单及异常流程的 MockMvc 测试	4	约 35%
Bug 根因分析	Codex / ChatGPT	输入复现现象、相关代码和期望行为，分析可能根因与修复方案	3	约 50%
演示说明整理	Codex / ChatGPT	根据 README、测试账号和主流程整理演示路径	4	约 20%

2. AI 助力分析

2.1 AI 在编码中最有价值的场景

AI 在编码阶段最有价值的是生成“第一版结构”：

1. 根据需求快速列出 Controller、Service、DTO、VO 和测试点。
2. 为重复性 CRUD 和列表查询提供基础实现思路。
3. 提醒常见异常分支，例如未登录、非参与方、重复提交、非法状态。
4. 帮助把 Bug 现象转化为可检查的字段、接口和状态变化。

2.2 AI 在测试生成中最有价值的场景

AI 对测试生成的帮助主要体现在测试场景枚举：

1. 能快速列出正常流程和异常流程。
2. 能提示状态机中不应跳过的状态。

- 3. 能提醒测试不仅要断言返回值，还要检查数据库或对象状态。
- 4. 能帮助把集成测试流程拆成可复用的登录、发任务、申请接单、确认接单辅助方法。

2.3 AI 在调试中最有价值的场景

AI 在调试中适合做“排查清单助手”：

- 1. 根据现象列出可能根因。
- 2. 提醒前端、后端、数据库三侧都要检查。
- 3. 对典型问题能快速给出方向，例如字段不一致、状态未同步、权限缺失、边界值未限制。
- 4. 帮助把复杂 Bug 拆成多个可验证的小问题。

3. AI 误导分析

3.1 AI 代码信任度实验数据摘要

指标	记录
实验功能	订单完成后的评价提交与信用流水记录
AI 直出测试通过率	初稿不能直接按项目结构运行
人工审查修复后测试通过率	可纳入服务层测试运行
发现的主要问题类型	业务边界不完整、信用分上下界缺失、测试断言不足、项目风格不一致

AI 的主要问题不是“完全不会写”，而是它容易写出看起来合理但没有完全贴合项目规则的代码。

3.2 AI 调试对决总结

项目	记录
AI 擅长调试的 Bug 类型	边界值、字段映射、接口契约、常见框架配置
AI 不擅长调试的 Bug 类型	跨模块状态机、历史数据保留、业务语义取舍
AI 修复方案的典型问题	倾向于快速隐藏现象，未充分区分当前态数据和审计数据

典型例子是服务方取消订单流程。AI 能指出“清空服务方、恢复任务状态、隐藏聊天评价”，但不会自然意识到“订单状态日志必须保留用于审计”。

4. Prompt 改进计划

4.1 上阶段改进措施的验证结果

上阶段提出要在 Prompt 中补充“模块背景、接口契约、数据库字段和验收标准”。P4 实践证明这项措施有效。

当 Prompt 只写“帮我实现评价功能”时，AI 输出偏通用；当补充订单状态、参与方规则、重复评价限制和信用流水要求后，AI 输出更接近项目实际。

4.2 编码阶段 Prompt 最佳实践

编码阶段比较有效的 Prompt 模板：

请基于以下项目上下文实现功能：

1. 当前技术栈：
 2. 已有文件和类：
 3. 业务规则：
 4. 必须保持的接口契约：
 5. 不允许改动的边界：
 6. 需要补充的测试场景：
- 请优先沿用项目现有风格，不要重新设计架构。

4.3 调试阶段 Prompt 最佳实践

调试阶段比较有效的 Prompt 模板：

现象：
复现步骤：
期望结果：
实际结果：
相关代码：
相关数据库状态：
已尝试的修复：
哪些数据必须保留：
请先分析根因，再给出最小修复方案和验证步骤。

4.4 下一阶段改进计划

1. 每次让 AI 改代码前，先给出相关文件路径和不可破坏的业务规则。
2. 对 AI 给出的修复方案先做人工审查，再执行。
3. 对状态机、信用分、权限和数据一致性问题，要求 AI 同时给出测试用例。
4. 对 AI 生成的测试，人工检查是否有关键断言，避免“空洞测试”。

5. 本阶段的核心工程判断

5.1 决策内容

本阶段最重要的工程判断是：

优先保证“发布需求 -> 申请接单 -> 确认接单 -> 订单执行 -> 完成评价”主链路稳定，而不是继续扩展更多 P2 功能。

5.2 为什么 AI 无法单独做这个决策

AI 可以根据需求文档建议功能优先级，但它不知道团队实际工时、成员熟悉程度、演示重点和验收风险。

例如 AI 可能建议继续补智能匹配、统计图表或更完整的 IM 能力，但这些功能会占用主链路调试时间。对 P4 来说，主链路稳定比扩展功能更重要。

5.3 团队最终如何判断和取舍

团队最终采用以下策略：

1. P0/P1 主流程必须可运行、可演示。
2. P2 功能只做基础预留或简单展示。
3. Bug 修复优先级高于新功能扩展。
4. 测试优先覆盖订单状态、权限、评价和后台治理。
5. 文档优先服务验收，确保交付物能对应到代码、测试和配置。

6. 阶段总结

P4 阶段的 AI 协作结论是：

AI 能显著提高编码、测试和调试的起步速度，
但不能替代团队对业务边界、状态一致性和交付优先级的判断。

在 CampusHub 项目中，AI 更像一个高效的副驾驶：它能帮我们更快发现路线，但最终方向盘仍必须由团队掌握。